

Модели на разпределени софтуерни архитектури. Среди и протоколи за разпределени приложения.

Ивайло Андреев + chat gpt 5.5, Claude Sonnet 4.6

29 юни 2026

Съдържание

1	Параметри на паралелната и разпределената обработка	3
1.1	Конкурентна, паралелна и разпределена обработка	3
1.2	Параметри на конкурентната обработка	3
1.2.1	Паралелизъм	3
1.2.2	Грануларност	3
1.2.3	Балансиране	4
1.2.4	Синхронизация	4
1.2.5	Тип на декомпозицията	4
1.3	Метрики	5
1.3.1	Ускорение и ефикасност	5
1.3.2	Суперлинейна аномалия	5
1.3.3	Нелинейна аномалия	5
1.3.4	Графика на ускорението и ефикасността	5
1.3.5	Цена на обработката	6
1.3.6	Закон на Амдал	6
1.4	Сравнение между паралелна и разпределена обработка	7
2	Модели на разпределените софтуерни архитектури и техните структури, организация, компоненти и приложение	7
2.1	[НЕ ЗАДЪЛБАВАЙ] Дефиниции	7
2.2	[НЕ ЗАДЪЛБАВАЙ] Процедурни модели	7
2.3	[НЕ ЗАДЪЛБАВАЙ] Обектни модели	7
2.4	Потокови модели	8
2.4.1	Видове	8

2.5	Контекстни (Data centric) модели	9
2.5.1	Типове	9
2.6	Йерархични модели	9
2.6.1	Слоест модел	10
2.6.2	Master/Slaves	10
2.7	Асинхронни модели	10
2.7.1	Producer/Consumer	10
2.7.2	Publisher/Subscriber	11
2.8	Интерактивни модели	11
2.8.1	Кратко обяснение (от консултацията)	11
2.8.2	Подробно обяснение	12
3	Организация на разпределените приложения	12
3.1	Клиент-сървър	12
3.1.1	[Наблегнато на консултацията] Трислойна архитектура	13
3.1.2	[НЕ наблегнато на консултацията] N-слойна архитектура	13
3.1.3	Брокерен модел	14
3.2	Peer-to-Peer архитектури	14
3.2.1	Неструктурирани P2P	14
3.2.2	Структурирани P2P	14
3.2.3	Хибридни / йерархични	14
3.2.4	Диаграми за екстра точки	15
3.3	Web-сървъри и сървъри за приложения	15
3.4	Метасистеми и грид	15
3.5	Сервизно-ориентирана архитектура (SOA)	16
3.6	Моделно-ориентирана архитектура (MDA)	16
3.7	Аспектно-ориентирана архитектура (AOP)	17
3.8	Софтуерни агенти	17

1 Параметри на паралелната и разпределената обработка

1.1 Конкурентна, паралелна и разпределена обработка

Дефиниция 1. Конкурентна обработка (Concurrency) е концепция, при която няколко задачи се изпълняват в застъпващи се времеви интервали. Тя определя структурата на софтуера и възможността му да управлява много процеси едновременно, без те задължително да се изпълняват в един и същ физически момент.

Дефиниция 2. Паралелна обработка (Parallel Computing) е физическото изпълнение на няколко задачи в един и същ момент от време. Тя изисква хардуер с множество изчислителни ядра или процесори в рамките на **една машина** (мултипроцесор със споделена памет), като основната цел е постигане на **максимално ускорение**.

Дефиниция 3. Разпределена обработка (Distributed Computing) е обработка на данни, при която задачите са разпределени физически върху **няколко независими машини (възли)**, свързани в мрежа (мултикомпютър). Всеки възел има собствена памет, а обмяната на данни се извършва чрез предаване на съобщения. Основните цели са **функционално разслояване**, висока надеждност и споделяне на ресурси.

1.2 Параметри на конкурентната обработка

1.2.1 Паралелизъм

Паралелизъм наричаме броя процеси (нишки), които изпълняват дадена задача. Той се дели на два типа - **софтуерен** и **хардуерен**. Софтуерният паралелизъм е характеристика на програмите - колко процеса (нишки) се стартират. Хардуерният паралелизъм зависи от броя процесори (вкл. ядра), върху които се изпълнява задачата. Ускорението от софтуерния паралелизъм е ограничено от хардуерния - не можем да очакваме 8 пъти ускорение от 8 нишки, ако машината има само 2 ядра.

Обикновено паралелизмът се отбелязва с p .

1.2.2 Грануларност

Грануларност е степента на декомпозиция на данните, т.е. на колко подзадания се разбива цялата задача. Различават се три вида грануларност, като те се характеризират по ефекта си, а не по конкретни числа (защото числата са различни за различните задачи):

- **едра** - малък брой подзадачи. В този случай малко време се прекарва в разпределение, но баланса е лош.

- **фина** - много голям брой подзадачи. Балансът на задачите е много добър - всеки процес получава еднакъв брой *операции*, но разпределянето на задачите отнема много време и забавя значително системата.
- **средна** - компромис между горните две. Това е грануларността, към която се стремим.

1.2.3 Балансиране

Както казахме, **балансирането** се грижи всеки процес да получи еднакво количество *работа*. Това е желано, защото времето за работата на паралелен алгоритъм е времето на приключване на **най-бавния** процес (нишка). Следователно е нежелано един процес да приключи работа, докато друг е претоварен със задания. Балансирането можем да разделим на **статично** и **динамично**, в зависимост от това кога се случва. Динамично балансиране е удачно (и дори наложително), когато нови задания постъпват по време на изпълнение. Можем да разделим балансирането и по начина, по който се осъществява:

- централно
- разпределено (напр. верига, решетка, хиперкуб)
- йерархично

1.2.4 Синхронизация

Синхронизацията описва **обмена на информация** между процесите и как той се случва. Можем да разделим алгоритмите на три вида според нея:

- **асинхронни** - не се обменят данни между процесите
- **локално синхронни** - всеки процес си има дефинирани съседи, с които обменя информация
- **глобално синхронни** - всеки процес си говори със всеки друг (пълен граф)

1.2.5 Тип на декомпозицията

Декомпозицията на задачата може да е по данни и по управление. В първия случай имаме **SPMD (Single Program, Multiple Data)**, а във втория - **MPMD (Multiple Program, Multiple Data)**. Тази характеристика е ортогонална на това дали ползваме една или много машини - и двата типа алгоритъма могат да се прилагат на двата случая. Разликата е, че при **SPMD** ползваме една и съща програма, която стартираме върху различните данни. В другия случай имаме няколко различни програми.

1.3 Метрики

1.3.1 Ускорение и ефикасност

Ускорение (*acceleration*) пресмятаме чрез $S_p = \frac{T_1}{T_p}$, където T_1 е времето за серийна обработка, а T_p е времето за паралелна обработка при степен на паралелизъм p . Поради наличие на комуникационни и синхронизационни закъснения обикновено $S_p \in (1, p)$ - ще разгледаме контрапримери след малко.

Ефективност (*efficiency*) е нормализирано ускорение: $E_p = \frac{S_p}{p}$, като съответното обикновено стойностите са до 100%.

1.3.2 Суперлинейна аномалия

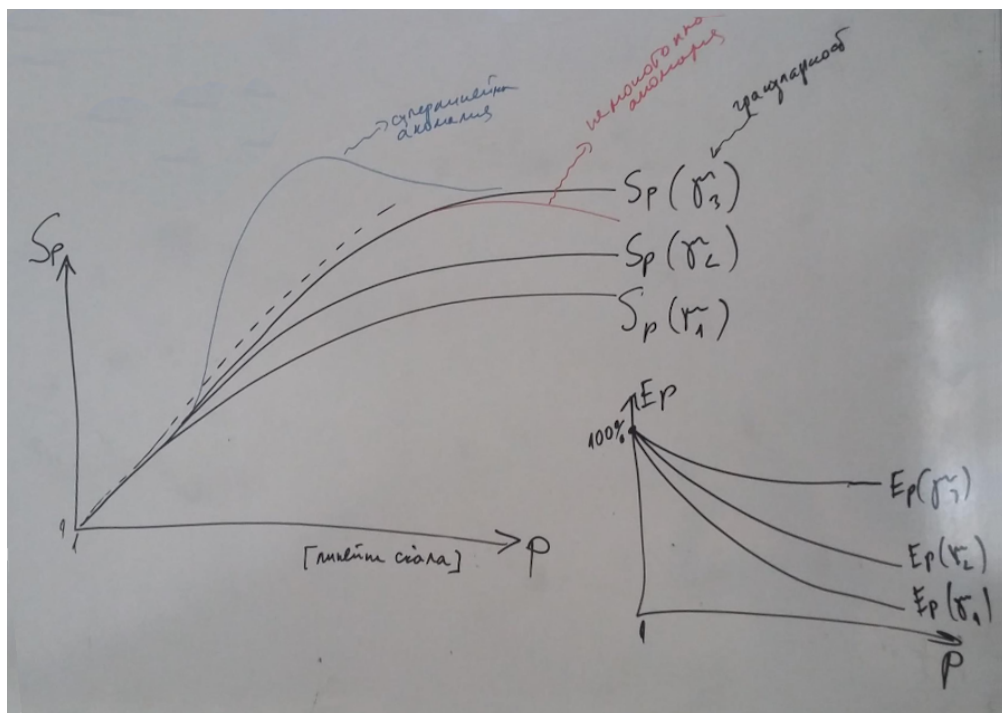
Този тип аномалия настъпва, когато получим $S_p > p$. Обикновено се случва заради кеша на данните. При пускане на повече нишки може да се включат повече хардуерни ядра, като всяко от тях има собствен **L1** кеш. Допълнително, ако областите от данни се припокриват между нишки, то дадено парче данни се зарежда само веднъж от бавната оперативна памет, и след това се достъпва бързо от споделения **L2** кеш.

1.3.3 Нелинейна аномалия

Това се случва когато получим $S_{p+1} < S_p$ за някое p , т.е. програмата се забавя при увеличаване на паралелизма. Причините за това може да са алгоритмични, или просто сме надвишили хардуерния паралелизъм, заради което трупаме единствено време за **context switching**, балансиране и разпределяне, но работата не се върши по-бързо.

1.3.4 Графика на ускорението и ефикасността

Обикновено за дадена система се рисуват фамилия криви ускорение (и/или ефикасност), в зависимост от определен параметър. Най-често този параметър е гранулярността. Ще покажем примерни графики, които включват и двете аномалии.



1.3.5 Цена на обработката

Цена (*cost*) е сумарното процесорно време на всички процеси заедно:

$$C_p = T_p \cdot p$$

Допълнителната работа (overhead) спрямо серийното изпълнение е:

$$W = C_p - T_1 = T_p \cdot p - T_1$$

Желаемо е W да е малко. При идеален алгоритъм ($W = 0$) имаме $C_p = T_1$, т.е. не се губи процесорно време за комуникация и синхронизация.

1.3.6 Закон на Амдал

Всеки алгоритъм се състои от серийна част и част, която може да бъде паралелна. Това налага горна границата на възможното ускорение, която се изразява чрез **закон на Амдал**:

$$S(n) = \frac{1}{(1 - \alpha) + \frac{\alpha}{n}}$$

където:

- $S(n)$ е теоретичното ускорение на изпълнението на цялата задача.
- n е степента на паралелизъм (брой процесори/нишки).
- α е фракцията от времето на задачата заемано от секцията, която сме паралелизирали, при серийно изпълнение на алгоритъма.

1.4 Сравнение между паралелна и разпределена обработка

Параметър / Тип обработка	Паралелна	Разпределена
Инфраструктура	мултипроцесор	мултикомпютър
Обмен на данни	Обща памет и механизми за синхронизация	Предаване на съобщения чрез middleware
Обичайна декомпозиция	SPMD	MPMD
Примери за синхронни задачи	nBody, WaTor	Client-server, P2P, някои конвейери
Примери за асинхронни задачи	фрактали	някои конвейери

2 Модели на разпределените софтуерни архитектури и техните структури, организация, компоненти и приложение

2.1 [НЕ ЗАДЪЛБАВАЙ] Дефиниции

Софтуерната архитектура дефинира какви са съставните процеси на една софтуерна система, как те са структурирани и си взаимодействат. *Моделите на софтуерни архитектури* представляват богати на информация и точни диаграми, описващи дизайна на системата. Под дизайн се разбира съвкупността от декомпозицията на съставните компоненти, техните функции, прилагания архитектурен стил и качествени атрибути.

2.2 [НЕ ЗАДЪЛБАВАЙ] Процедурни модели

Процедурните архитектурни модели се базират на разпределена обработка чрез *отдалечено извикване на процедури (remote procedure call (RPC))* или *Remote Method Invocation (RMI)*. Представява извикване на процедура на един възел от друг възел, като това е прозрачно за извикващия възел, т.е. е като локална за него процедура.

2.3 [НЕ ЗАДЪЛБАВАЙ] Обектни модели

Обектно-ориентираните архитектурни модели се основават на разделянето на отговорностите на системата в индивидуални повторно използвани и независими обекти, които си сътрудничат. Основните принципи на тези модели са:

- *Абстракция* - опростяването на решението на реален проблем чрез дефиниране на подходящи класове.

- *Композиция* - изграждането на обекти от други обекти.
- *Наследственост* - автоматичното присвояване на функционалностите на едни обекти на други обекти и тяхното разширение и промяна.
- *Капсулация* - скриването на ненужните за потребителя детайли на някои обекти.
- *Полиморфизъм* - присвояването на едно и също име на различни операции на клас или йерархия от класове.

2.4 Поточкови модели

Потоковите архитектурни модели разглеждат системата като последователност от трансформации на последователен набор от данни. Всеки компонент трансформира входните си данни в изходни. Топологията на пренос на данните между тях се задава експлицитно чрез **блок-диаграми**. Модулите поддържат **само** интерфейс по данни, но не и контролен, като те не се адресират директно, а чрез предаваните данни.

2.4.1 Видове

Можем да разграничим няколко вида потокови архитектури в зависимост от това кога и как се извършва обработката на данните от всеки модул:

- *Пакетна обработка (Batch Sequential)* - Модулите се извикват в последователен ред един след друг, като изходът от един модул се предава като вход на следващия. Пакетите от данните, които се обработват от всеки модул, са записани под формата на **временни файлове**, които служат за комуникация между два последователни в изпълнението си модули. Ключовото е, че обработката от следващия модул може да бъде **отложена във времето**. Най-често този модел се използва за асинхронна обработка на данни, като се цели оптимално използване на ресурси за сметка на паралелизъм и интерактивност. **Примери:** профилактика на системата; безплатна обработка, която се случва само когато системата не е натоварена от платени потребители.

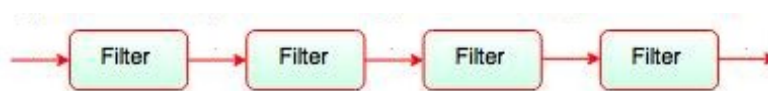


Fig. Batch Sequential

- *Филтрирани канали (Pipe and Filter)* - Обработката се случва в онлайн режим. Всеки процес (отговарящ на модул) е активен и е свързан директно със съседите си. Всеки процес започва да предава изходните данни към следващия филтър преди да е свършил с обработката на всичките данни. Този начин за обработка на данни позволява паралелизъм (конвейеризация).

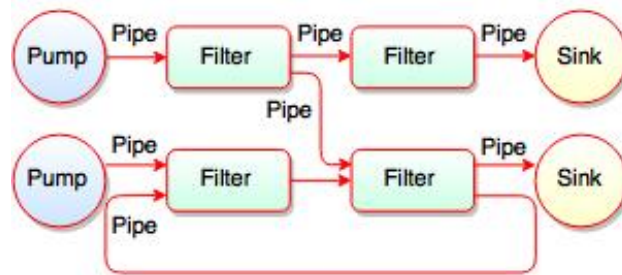


Fig. Pipes and Filters

- *Контролни процеси (Process Control)* - като *Pipe and Filter*, но с добавени ограничения върху времето за изпълнение (**realtime constraints**). Понякога между модулите се добавя и пряк физически интерфейс. Както при всяка система в реално време, целта е да се гарантира време за изпълнение. **Примери:** управление на физически процеси, например при автомобилите и самолетите.

2.5 Контекстни (Data centric) модели

Контекстните архитектурни модели се характеризират с централизирано хранилище за данните, от където те са достъпни за всички останали компоненти на системата. Декомпозицията на модулите се състои от модул за **управление на достъпа до данните** и **агенти**, които извършват операции върху тях. Интерфейсът между агентите и данните може да бъде явен (при **RMI** и **RPC**) или имплицитен (напр. **транзактивен**). В оригинал не се предвижда комуникация между агентите.

2.5.1 Типове

Съществуват два основни контекстни модела:

- *Хранилище (Repository)*, където **агентите** са активни, т.е. те инициират взаимодействието. Примери са СУБД, CORBA и UDDI.
- *Черна дъска (Blackboard)*, където инициативата е на **модула с данните**, а агентите се абонират за събития (*event listeners*). Събитие настъпва при промяна в данните. Използват се при AI и мултидисциплинарните системи.

2.6 Йерархични модели

Йерархичните архитектурни модели декомпозират системата на йерархични модули, като функциите се групират по йерархичен принцип на няколко нива. Координацията обикновено е между модули от различни нива (вертикална свързаност) и се базира на явни (т.е. "заявка-отговор") обръщания. Слоевете делегират

своята работа на услуги от съседни по-ниски нива. Пълна прозрачност между нивата се постига при запазване на свързващите интерфейси.

Ще разгледаме два основни типа йерархични модели

2.6.1 Слоест модел

При този модел, имаме последователност от слоеве, като всеки предоставя услуги на по-горните. Използван при много ОС като Unix, GNU и Windows. Също така протоколните стекове OSI и TCP/IP са слоести архитектури. Целта на този тип архитектури е **бързо проектиране на (преносими) приложения**.

2.6.2 Master/Slaves

Master/Slaves представлява разбиване на главна програма и подчинени програми. Главната нарежда на подчинените какво да правят. Този модел може да цели две неща:

- **отказоустойчивост и надеждност:** постига се, като главната програма проверява резултата от робите, както и статуса им
- **ускорение:** постига се чрез балансиране на натоварването между робите за по-ускорено изпълнение

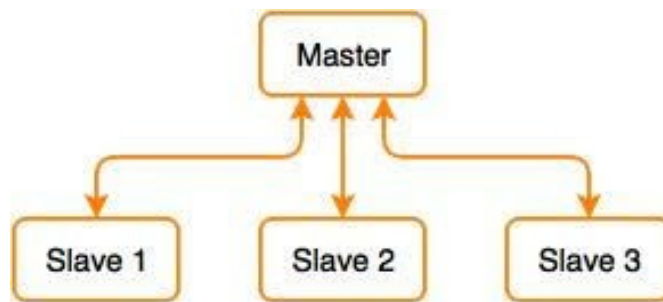


Fig. Master-Slave Architecture

2.7 Асинхронни модели

Асинхронните модели се базират на имплицитни асинхронни обръщания между обслужващите процеси, т.е. обмяна на съобщения - **message passing**.

2.7.1 Producer/Consumer

Асинхронният обмен може да бъде от тип **Producer/Consumer**, където обменът на данни е 1:1. Има следните разновидности:

- *интерактивен (online)*, където няма буфериране и и двата процеса са активни. Често се ползва **адресна книга**, която позволява процесите да се адресират по име, а не да трябва да знаят своя мрежови адрес - например **DNS**.
- *отложен (offline)*, където обмена на съобщенията се опростява посредством **процес-буфер**, който служи като пощенска кутия. Това позволява консуматора да не е активен по време на изпращане на съобщения, както и обратното.

2.7.2 Publisher/Subscriber

Publisher/Subscriber също е вид асинхронен обмен на съобщения където обменът на данни е 1:*. Има следните разновидности:

- *topic-based*, където съобщенията се изпращат до topic-и, които са именувани канали. Издателят е отговорен за дефинирането на topic-ите, а абонатите получават съобщенията от всички topic-и, за които са се абонирали.
- **[Не е споменато на консултацията]** *content-based*, където съобщенията се изпращат до абонат, ако атрибутите на съдържанието отговарят на ограниченията дефинирани от абоната. Абонатът е отговорен за класификацията на съобщенията.

2.8 Интерактивни модели

Интерактивните модели поддържат интензивен потребителски интерфейс. За целта декомпозицията на системата е на 3 функционални модула:

- *модул за представяне (изглед)*, имплементиращ потребителския интерфейс. Използва се за представяне (в т.ч. графично и мултимедийно) на изходните данни и намеса на потребителите в обработката (т.е. вход за данни и контрол)
- *модул данни*, поддържащ данновия модел заедно с базова функционалност за обработката на данните
- *модул за управление*, който отговаря за системни комуникации, управление на процесите, инициализиране и конфигуриране на модули данни, както и управление на изгледи.

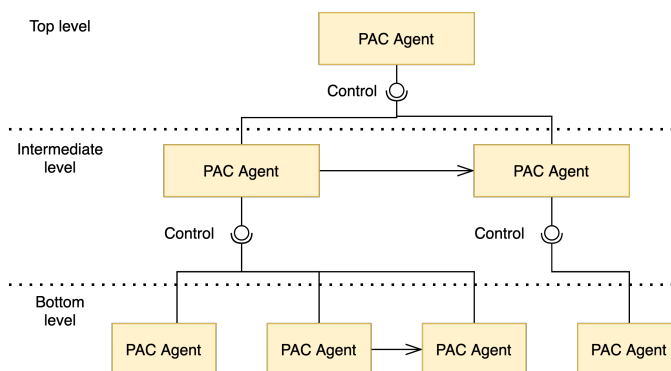
2.8.1 Кратко обяснение (от консултацията)

Гореописаните компоненти се имплементират от **MVC (Model View Controller)**. **PAC (Presentation-Abstraction-Control)** е MVC за многоагентна система, например ако има много източници на данни. Всеки агент изпълнява всеки от PAC атрибутите.

2.8.2 Подробно обяснение

Има две категории интерактивни софтуерни архитектури - **PAC (Presentation-Abstraction-Control)** и **MVC (Model-View-Controller)**, като компонентите Presentation, Abstraction и Control в PAC съответстват приблизително на View, Model и Controller в MVC. Те се различават по начина на управление като:

- **PAC** е с йерархично (разслоено) и разпределено управление, при което системата се формира от набор коопериращи агенти на три нива - **базово ниво**, състоящо от агенти работещи върху общи данни и бизнес логика, **средно ниво**, състоящо от агенти координатори на изгледите и **ниво на изгледите** за обработка на локални данни. Всеки агент имплементира **P, A и C** компоненти.



- при **MVC** агентите са равнопоставени.

3 Организация на разпределените приложения

При наличие на множество потребители, като системи за масово обслужване, се използва **MPMD** декомпозиция.

3.1 Клиент-сървър

Клиент-сървър (или двуслоен/two-tier архитектура) е класически **MPMD** модел за адаптиране на ограничена клиентска инфраструктура към сложни задачи. Процесите се класифицират като два основни:

- **Сървър**, който е реактивен процес, имплементиращ приложната логика и данния модел под формата на конкретен набор от услуги.
- **Клиент**, който е активен интерфейсен процес, използващ услугите на сървъра.

Сървърите са **специално проектирани** устройства, направени да обработват определен товар. Те са свързани с **опорни мрежи (backbone)**. Клиентските устройства (които наричаме и клиентска инфраструктура), както и връзките им с мрежата, са произволни.

Има два основни вида клиенти - **тънки (thin)**, които делегират цялата работа на сървъра, и **дебели (fat)**, които изпълняват част от задачата.

Предимства	Недостатъци
технологично специализиране	унификация на клиентската инфраструктура
инфраструктурна гъвкавост (скалиране)	по-трудна защита на достъпа до данните
преизползване	скалируемост на сървъра
еволюция без участието на потребителя	скъпа поддръжка, профилактика и тестване

Клиент-сървър моделът е широко разпространен в Интернет приложенията и Web архитектурите, изградени върху **TCP/IP** протоколния стек.

3.1.1 [Наблегнато на консултацията] Трислойна архитектура

При *три-слойната архитектура* сървърната част се декомпозира допълнително на **сървър на приложението**, в което се обработват заявките, и **сървър на данните**. Основната цел е допълнително функционално разслояване.

3.1.2 [НЕ наблегнато на консултацията] N-слойна архитектура

N-слойната архитектура включва надграждане над клиент-сървър архитектурата, където компонентите са разделени на няколко слоя. Можем да си представим, че слоевете са вертикално подредени и компонентите на даден слой са клиенти на сървъри от съседния долен слой.

При **N-слойните архитектури** редовно се срещат **многослойни сървъри**, където сървърната част е декомпозирана на поне два слоя, които се класифицират като:

- *междинни слоеве*, които се намират между интерфейския и вътрешния слой, имплементиращ "бизнес логиката".
- *вътрешен слой*, най-долният слой, който обичайно е персистентен или се използва за комуникации.

Очевидно щом е надграждане над клиент-сървър архитектурата, то *N-слойната* разполага със същите недостатъци и предимства, но по-силно изразени.

3.1.3 Брокерен модел

При този модел се добавя допълнителен модул - **брокер**. Той предоставя **функционално еквивалентни** алтернативи на дадена услуга. Позволява и услугите да се класират по своите **функционални** и **нефункционални** атрибути.

Пример: Брокер за принтиране, на който можем да кажем че искаме принтирането да е черно-бяло (функционално) и да се извърши от принтера, с най-къса опашка (нефункционално).

3.2 Peer-to-Peer архитектури

Peer-to-Peer (P2P) архитектурата се разгръща като **наслоена (overlay) мрежа** върху съществуваща клиентска част от мрежова инфраструктура. Всеки участник е еднакво привилегирован и заделя част от ресурсите си (сри, диск и др.), която да се използва от останалите участници в мрежата без нуждата от каквато и да е централна координация. За разлика от класическата архитектура клиент-сървър всеки участник изпълнява функцията на клиент и сървър едновременно за другите участници. **P2P** се характеризира чрез **децентрализирана топология**, **самоорганизация**, **инцидентна (ad hoc)** и **динамична свързаност и наличност** на възлите и процесите в тях, **скалируемост**, **отказоустойчивост (fault tolerance)**, **анонимност** на споделянето и специална юридическа регулация.

3.2.1 Неструктурирани P2P

При тях топологията на свързване е произволна, както и производителността на комуникационните канали. Често се случва възли да се включват и отпаднат. В резултат, общата производителност е също произволна.

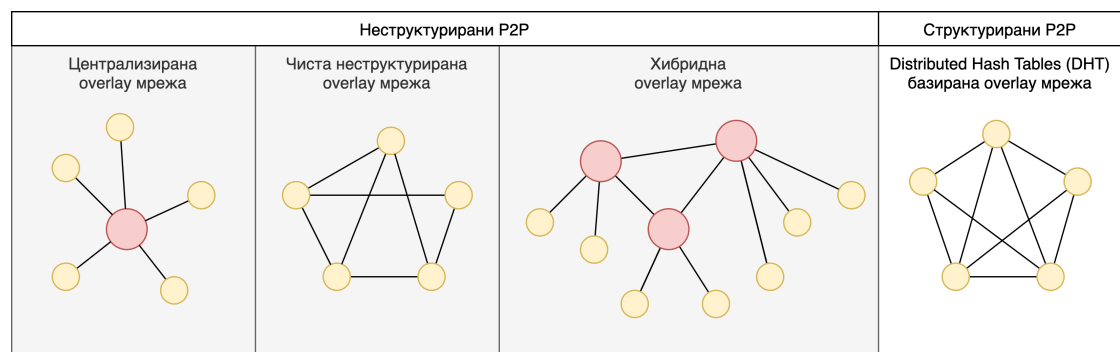
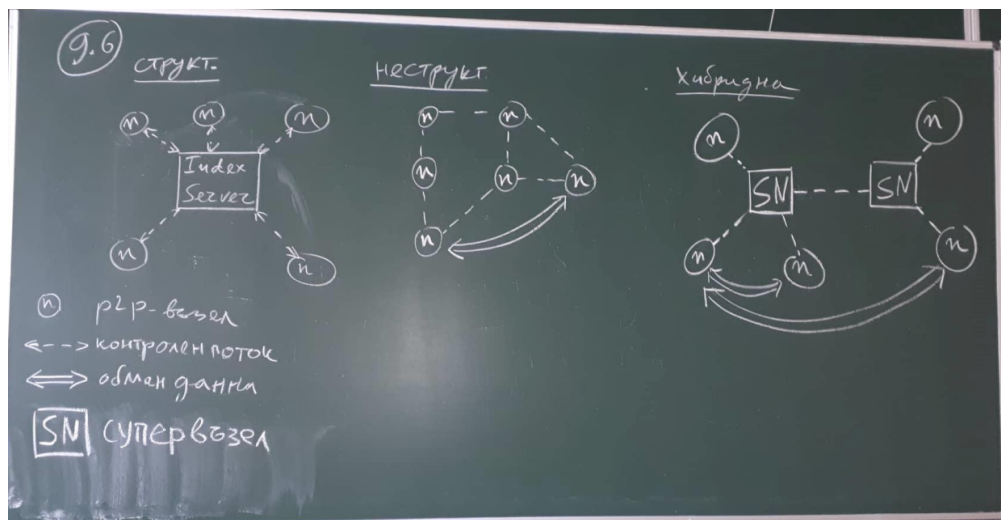
3.2.2 Структурирани P2P

Топологията е изрично зададена, което дава гаранции и за бързодействието. Съществуват няколко топологии, като **хорда** е много популярна. При нея всеки възел се връзва с фиксиран брой съседни.

3.2.3 Хибридни / йерархични

Ниските нива са структурирани, като всяко ниско играе като супервъзел в неструктурирана мрежа.

3.2.4 Диаграми за екстра точки



3.3 Web-сървъри и сървъри за приложения

Web-сървърът обслужва HTTP/HTTPS заявки и предоставя предимно **статично съдържание** (HTML, CSS, JS, изображения). Може да препраща динамични заявки към сървър за приложения. **Примери:** Apache HTTP Server, Nginx, Microsoft IIS.

Сървърът за приложения (Application Server) предоставя среда за изпълнение на **бизнес логиката** на приложението. Той управлява нишки, транзакции, connection pool и сигурност. **Примери:** Apache Tomcat, WildFly, GlassFish, WebLogic, WebSphere.

В трислойна архитектура Web-сървърът образува презентационния слой, сървърът за приложения — логическия, а базата данни — персистентния слой.

3.4 Метасистеми и грид

Грид-изчисленията (Grid Computing) обединяват изчислителните ресурси (процесори, памет, диск) на много машини — евентуално от различни организации

— в единна виртуална суперизчислителна система. Използват се за задачи, изискващи огромна изчислителна мощ (научни симулации, биоинформатика и др.).

Характеристики:

- **Хетерогенност** — различни хардуерни и софтуерни платформи
- **Географска разпределеност** — ресурси в различни места и организации
- **Динамично предоставяне на ресурси** — resource provisioning при нужда
- **Middleware за управление** — напр. Globus Toolkit

Примери: BOINC, SETI@home, TeraGrid, EGEE.

Метасистемите са системи, обединяващи множество хетерогенни изчислителни системи под общ интерфейс, като скриват разнородността от потребителя.

3.5 Сервизно-ориентирана архитектура (SOA)

Сервизно-ориентираната архитектура (SOA) е архитектурен стил, при който функционалността е декомпозирана на **слабо свързани услуги (services)**, достъпни чрез стандартизирани интерфейси по мрежата.

Основни принципи: слабо свързване, абстракция, повторна употреба, автономност, откриваемост.

Ключови компоненти:

- **WSDL** (Web Services Description Language) — описва интерфейса на услугата
- **SOAP** (Simple Object Access Protocol) — XML-базиран протокол за обмен на съобщения
- **UDDI** (Universal Description, Discovery and Integration) — регистър за откриване на услуги
- **ESB** (Enterprise Service Bus) — шина, координираща комуникацията между услугите

Ролите в SOA са три: **доставчик** (публикува услугата), **регистър** (UDDI) и **потребител** (открива и извиква услугата).

3.6 Моделно-ориентирана архитектура (MDA)

Моделно-ориентираната архитектура (MDA — Model-Driven Architecture) е подход, при който **моделите** са първичните артефакти на разработката, а кодът се генерира автоматично.

Нива:

- **CIM** (Computation Independent Model) — описва изискванията от бизнес гледна точка

- **PIM** (Platform Independent Model) — описва системата независимо от платформата
- **PSM** (Platform Specific Model) — описва системата за конкретна платформа

Трансформациите **PIM** → **PSM** → **код** се извършват (полу-)автоматично. Целта е преносимост и намаляване на ръчния код.

3.7 Аспектно-ориентирана архитектура (AOP)

Аспектно-ориентираното програмиране (AOP) отделя **crosscutting concerns** — функционалности, пресичащи много компоненти — в отделни модули, наречени **аспекти**.

Примери за crosscutting concerns: логване, сигурност, управление на транзакции, кеширане.

Основни понятия:

- **Aspect** — модул, капсулиращ crosscutting concern
- **Join point** — точка в изпълнението на програмата (напр. извикване на метод)
- **Pointcut** — набор от join points, в които се прилага аспектът
- **Advice** — кодът, изпълняван при достигане на pointcut (before/after/around)
- **Weaving** — процесът на вмъкване на аспектите в основния код

Инструменти: **AspectJ** (Java), Spring AOP.

3.8 Софтуерни агенти

Софтуерен агент е автономна програмна единица, способна да **възприема средата** и да **действа върху нея** с цел постигане на конкретни цели.

Основни свойства:

- **Автономност** — действа без директно управление от потребителя
- **Реактивност** — реагира на промени в средата в реално време
- **Проактивност** — инициира действия сам за постигане на целите си
- **Социалност** — комуникира и си сътрудничи с други агенти

Видове агенти: прости рефлексни, базирани на модел, базирани на цели, базирани на полезност.

Многоагентните системи (MAS) се състоят от множество взаимодействащи агенти. Приложения: роботика, AI системи, симулации, автоматизирана търговия, разпределено управление.